

SMART CAMPUS FACILITY & LAB BOOKING SYSTEM

Flipped Course Project Report - Programming in Java
Platform: VITyarthi | Academic Evaluation

Course: Programming in Java	Platform: VITyarthi
Student Name: Aniket Singh	GitHub Handle: thakuraniketsingh16-cloud
Architecture: Pure Java SE (Single File Main.java)	Status: 100% Tests Passed (Zero External JARs)

1. Abstract

Managing university shared infrastructure—including computing labs, seminar halls, auditoriums, and sports courts—requires automated conflict resolution and role-based policies. The **Smart Campus Facility & Lab Booking System** is a console-based Java application developed to demonstrate core Object-Oriented Programming (OOP) paradigms. It incorporates inheritance hierarchies, dynamic method dispatch, mathematical interval overlap conflict detection, and single-file flat database persistence.

2. Problem Statement & System Objectives

Ad-hoc manual booking leads to double-booking identical slots, students monopolizing computer labs without time limits, and unauthorized reservations of large auditoriums. This software enforces automated interval overlap validation, role-based session quotas, and permission checks.

3. Object-Oriented Architecture

- Facility Hierarchy:** Abstract class `Facility` extended by concrete classes `Lab` (workstations, OS, GPU), `Hall` (A/V, projector, AC), and `SportsCourt` (surface type, indoor/outdoor, floodlights).
- User Hierarchy:** Abstract class `User` extended by `Student` (restricted to 2-hour sessions) and `FacultyMember` (permitted up to 6 hours with auditorium booking authorization).
- Booking Entity:** Encapsulates reservation interval and conflict validation logic.

4. Core OOP Principles Applied

OOP Concept	Application in System
Abstraction	Abstract classes <code>Facility</code> and <code>User</code> define essential method contracts while concealing concrete implementation.
Inheritance	<code>Lab</code> , <code>Hall</code> , <code>SportsCourt</code> inherit from <code>Facility</code> ; <code>Student</code> , <code>FacultyMember</code> inherit from <code>User</code> .

Polymorphism	Dynamic method dispatch computes quota limits and displays specialized facility specs based on runtime instance.
Encapsulation	Private fields accessed exclusively via validated methods; interval overlap logic encapsulated in Booking.
Exception Handling	Custom exceptions (CampusBookingException, SlotUnavailableException, InvalidInputException) protect against conflicts.

5. Automated Test Suite Results

Test #	Test Case	Expected Result	Actual Result	Status
1	Facility & User Catalog Loading	Parse entities from data file	6 facilities, 5 users loaded	PASS
2	Standard Booking Creation	Create valid reservation	Booking B104 created (CONFIRMED)	PASS
3	Slot Collision & Overlap Check	Block double-booking same slot	SlotUnavailableException caught	PASS
4	Student Max Quota Enforcement	Reject 4-hour booking by student	InvalidInputException caught (limit 2 hrs)	PASS
5	Auditorium Booking Restriction	Block student from booking hall	CampusBookingException caught	PASS
6	Cancellation & Slot Re-allocation	Free slot on cancel & allow rebook	Slot re-booked successfully	PASS

Overall Score: 6 / 6 Tests Passed (100.0%)

6. Declaration of Originality

I declare that this project and its accompanying documentation are my original work submitted for the Programming in Java course on the VITyarthi platform. All code logic, OOP structures, and tests were written independently in compliance with academic guidelines.